ArkEdge Space - Eigenvalue Decomposition Demo Proposal

(Python → Verilog / SystemVerilog HDL Translation)

1. Overview

This document proposes a technical demo to explore how Code Metal's tools can be used to translate **high-level Python code into HDL (Verilog or SystemVerilog)**.

As a concrete example, we focus on **eigenvalue and eigenvector computation for complex Hermitian matrices**. Eigenvalue decomposition is a computationally intensive linear-algebra operation that appears in many advanced signal-processing and numerical-analysis pipelines, making it a suitable benchmark for automated HDL generation and hardware-oriented reasoning.

---

2. Objective

- **Input**: Hermitian matrix
- **Output**: Eigenvalues and eigenvectors
- **Deliverable**: RTL description (Verilog or SystemVerilog)

The nominal **target device** is **Xilinx Zynq UltraScale+ ZU7**.

However, we recognize that a full implementation may exceed available resources. Therefore, we welcome alternative implementation proposals, for example:

- A full-precision implementation with resource utilization analysis
- A resource-constrained implementation targeting ZU7, with:
  - Reduced numerical precision
  - Performance vs. accuracy trade-off analysis

The goal is not only functional correctness, but also to understand the **design decisions and trade-offs** made during translation.

---

3. Implementation Concerns & Discussion Points

3.1 RTL Mapping of NumPy Operations

- The reference Python implementation relies heavily on **NumPy functions**.
- Manually rewriting these operations in RTL is typically labor-intensive.
- Key discussion points:
  - Choice of arithmetic representation:
    - Fixed-point
    - Floating-point
    - Integer
  - Achievable numerical precision with each representation
  - Impact on:
    - Resource usage (DSPs, BRAM, LUTs)
    - Latency and throughput

The Python code below implements eigenvalue decomposition using the **Jacobi method**, but still contains many NumPy-dependent operations.

3.2 Challenges of the Jacobi Method

The Jacobi method was selected because it can compute eigenvalues and eigenvectors simultaneously.

However, it also presents challenges for hardware implementation:

- Convergence can be slow.
- The number of iterations required is data-dependent and not known in advance.
- The algorithm involves repeated global matrix updates, which complicates:
  - Deterministic scheduling
  - Pipelining
  - Overlapping computation with incoming data streams

Typical operations involved include:

- Maximum-value search
- Division

- Square root
- Trigonometric and inverse trigonometric functions

These operations are not always a natural fit for FPGA-based implementations.

For this reason:

- The demo is **not strictly limited to the Jacobi method**.
- Alternative algorithms are acceptable if they provide sufficient accuracy and improved hardware characteristics.

3.3 Alternative Algorithmic Approaches (Open for Proposal)

As part of the demo, we are also interested in exploring whether algorithmic restructuring can simplify hardware implementation. Possible directions include:

Householder Transformation + Subsequent Eigenvalue Solvers

- Reduction of a Hermitian matrix to tridiagonal form using Householder transformations
- Fixed iteration count and predictable scheduling
- Limited operation set (primarily division and square root)
- Improved suitability for pipelining and staged hardware implementation

After tridiagonalization, eigenvalue computation can be reduced to lower-order polynomial problems, with potential use of iterative refinement (e.g., Newton-based methods).

These approaches are mentioned as examples, not strict requirements.

---

4. Assumptions and Constraints

Input Characteristics

- **Matrix size**: 10 × 10 Hermitian matrix
- **Data format** (one of the following):
  - 46-bit signed integer (complex)
  - 32-bit single-precision floating point (complex)

Throughput Requirement

- One matrix is provided every 1 µs
  - 100 MHz → 100 clock cycles
  - 200 MHz → 200 clock cycles
- Target throughput: 1 eigenvalue decomposition per microsecond

Algorithm Choice

- The provided reference code uses the Jacobi method.
- The implementation is not required to use Jacobi, as long as:
  - Eigenvalues and eigenvectors are computed
  - Accuracy is sufficient for downstream numerical processing

---

5. Reference Python Program

The following Python code is provided as a functional reference and translation target. It avoids calling `np.linalg.eig` directly and instead implements eigenvalue decomposition explicitly, making algorithmic structure visible.

```python
import numpy as np

# Translation Target Function
def jacobi_eigen(A, max_iter=2000):
    n = A.shape[0]
    V = np.eye(n, dtype=complex)

    for z in range(max_iter):
        off_diag_max = 0
        p, q = 0, 1
        for i in range(n):
            for j in range(i + 1, n):
                if abs(A[i, j]) > off_diag_max:
                    off_diag_max = abs(A[i, j])
                    p, q = i, j

        if off_diag_max < 1e-8:
```

```
            break

        a_pp, a_qq, a_pq = A[p, p], A[q, q], A[p, q]
        theta = 0.5 * np.arctan((2 * np.abs(a_pq)) / (a_pp - a_qq))
        phi = -np.angle(a_pq)

        U = np.eye(n, dtype=complex)
        U[p, p] = np.cos(theta)
        U[p, q] = np.sin(theta) * np.exp(-1j * phi)
        U[q, p] = np.sin(theta) * np.exp(1j * phi)
        U[q, q] = -np.cos(theta)

        A = U.conj().T @ A @ U
        A[p, q], A[q, p] = 0, 0
        V = V @ U

    return z, np.diag(A), V
# Translation Target End

def show(A, tol=1e-10):
    B = A.copy()
    B.real[abs(B.real) < tol] = 0
    B.imag[abs(B.imag) < tol] = 0
    np.set_printoptions(precision=2, suppress=True)
    print(B)
    print()

np.random.seed(0)

N = 10
BW = 14
vect = np.random.randint(-2**BW, 2**BW, (N, 100)) + \
       np.random.randint(-2**BW, 2**BW, (N, 100)) * 1J

acm = vect @ vect.conj().T
ite, eigenvalues, eigenvectors = jacobi_eigen(acm.copy())
eig_val, eig_vec = np.linalg.eig(acm)

print(eigenvalues.real)
print(eig_val.real)

# print()
# show(eigenvectors)
```

```
# print()
# show(eig_vec)
```

Design Freedom and Evaluation Philosophy

We are not prescribing a specific algorithm for this task. Rather, we are interested in understanding how your tool reasons about hardware-friendly implementations for this class of numerical problems.

In particular, we welcome: - Algorithmic transformations that improve determinism, scheduling, or pipelining - Alternative formulations that reduce operator complexity or resource usage - Explicit discussion of trade-offs between precision, performance, and hardware cost

The proposed Jacobi-based Python implementation should be treated as a reference and starting point, not a constraint.

---

6. Expected Outcome of the Demo

The goal of this demo is to better understand:

- How much of the Python-based workflow can be automatically translated
- The quality, structure, and readability of the generated HDL
- Trade-offs between:
  - Numerical precision
  - Hardware resource usage
  - Performance
- Whether the tool can reason not only about syntax translation, but also about hardware-oriented algorithm design choices